KNOWLEDGE INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

SALEM – 637 504

DEPARTMENT

OF

ELECTRONICS AND COMMUNICATION ENGINEERING

LABORATORY RECORD

Course Code	**::**	BE23EE511
Course Name	**::**	EMBEDDED C PROGRAMMING
Year / Sem	**::**	III / V
Academic Year	**::**	2025 – 2026 (Odd Sem)
Regulation	**::**	R2023

Name of the Student	**::**	
Register Number	**::**	

KNOWLEDGE INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

[Approved by UGC & AICTE, Affiliated to Anna University Chennai]

NH – 544, KIOT Campus, Kakapalayam (PO), Salem – 637 504.

RECORD NOTEBOOK

REGISTER NUMBER

Certified that this is the bonafide record of work done by Selvan / Selvi ……..……of the **V** Semester **B.E.–Electronics and Communication Engineering** Branch during the year **2026 – 2027** in the **BE23EE511 - Embedded C Programming Laboratory.**

Course Faculty

Head of the Department

Submitted for the End Semester Practical Examination held on …….

INTERNAL EXAMINER

EXTERNAL EXAMINER

Knowledge Institute of Technology
(An Autonomous Institution)
Salem – 637 504

Vision and Mission of the Institution

Vision

To be a world class institution to impart value and need based professional education to the aspiring youth and carving them into disciplined world class professionals who have the quest for excellence, achievement orientation and social responsibilities.

Mission

- To promote academic growth by offering state-of-art undergraduate, postgraduate and doctoral programs and to generate new acknowledge by engaging in cutting-edge research.
- To nurture talent, entrepreneurship, all-round personality and value system among the students and to foster global competitiveness among students.
- To undertake collaborative projects which offer opportunities for long-term interaction with academia and industry.
- To purse global standards of excellence in all our endeavours namely teaching, research, consultancy, continuing education and support functions

Vision and Mission of the Department

Vision

To produce competent Electronics and Communication Engineers by imparting quality education to meet the industry requirements and for serving the societal needs

Mission

M1: To develop appropriate facilities for promoting research activities

M2: To inculcate leadership qualities among students for self and societal growth

M3: To nurture students on emerging technologies for serving industry needs through industry institute interface

M4: To enrich teaching learning process by transforming young minds to be resourceful engineers

Electronics and Communication Engineering	
Vision	
✓ To produce competent Electronics and Communication Engineers by imparting quality education to meet the industry requirements and for serving the societal needs	
Mission	
✓ To develop appropriate facilities for promoting research activities ✓ To inculcate leadership qualities among students for self and societal growth ✓ To nurture students on emerging technologies for serving industry needs through industry-institute interface ✓ To enrich teaching-learning process by transforming young minds to be resourceful engineers	
Programme Educational Objectives (PEOs) The graduates of Electronics and Communication Engineering will be able to	
PEO – 1	To enable graduates to pursue research, or have a successful career in academia or industries associated with Electronics and Communication Engineering, or as entrepreneurs
PEO – 2	Design, simulate and develop new innovative product and system in multi-disciplinary fields through applying life-long learning skill and modern tools handling ability
PEO – 3	Demonstrate communication skill, leadership qualities, ethics, team work and social responsibilities
Programme Specific Outcome (PSOs) The graduates of Electronics and Communication Engineering will be able to	
PSO – 1	Use signal processing concepts and tools to provide solutions to real time problems
PSO – 2	Use embedded system concepts for developing IoT applications
PSO – 3	Use the concepts of analog and digital electronics to design and implement VLSI circuits

Programme Outcomes (POs) The graduates of Electronics and Communication Engineering will be able to:	
PO – 1	✓ **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO – 2	✓ **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO – 3	✓ **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO – 4	✓ **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO – 5:	✓ **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO – 6:	✓ **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO – 7:	✓ **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO – 8:	✓ **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO – 9:	✓ **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO – 10:	✓ **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO – 11:	✓ **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO – 12:	✓ **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Outcomes:

At the end of the course, the student should be able to

CO1	Explain the basic structure and tools for embedded systems development.
CO2	Write modular C and Embedded C programs for handling embedded peripherals and memory.
CO3	Develop drivers for GPIO, UART, SPI, and handle interrupts in real-time scenarios.
CO4	Design interrupt-driven and RTOS-based systems with proper testing and debugging practices.
CO5	Integrate sensors, actuators, and protocols in a real-time embedded project.

GENERAL LABORATORY SAFETY RULES

The Electronic & Communication Engineering (ECE) Department is committed to providing a safe laboratory environment for all students. The ECE department requires the students, staff, and faculty to adhere to basic General Laboratory Safety Rules in all department operated labs halls.

Using laboratory equipment to perform any kind of experiments contains some elements of risk. Safe working habits are essential for your safety and the safety of others around you.

GENERAL LABORATORY SAFETY RULES

- ✓ Shoes shall be worn that provide full coverage of the feet and appropriate personal clothing shall be worn in laboratories.
- ✓ Students shall be familiar with the locations and operation of safety and emergency equipment such as, emergency power off, emergency telephones, and emergency exits.
- ✓ Do not displace or remove laboratory equipment without instructor or technician authorization.
- ✓ Never open or remove cover of equipment in the laboratories without instructor authorization.
- ✓ Electronic gadgets are prohibited in all ECE laboratories.

ELECTRICAL SAFETY GUIDELINES

- ✓ All circuit connections shall be verified by the instructor before switching ON the power supply.
- ✓ Power supply shall be switched OFF before making or modifying connections.
- ✓ Voltages above 50V AC / 50V DC are considered dangerous and must be handled with care.
- ✓ Students shall not touch live circuits with bare hands.
- ✓ Only regulated power supplies (5V / 3.3V) shall be used for microcontroller systems.
- ✓ Proper grounding must be ensured for all circuits.
- ✓ Damaged wires, connectors, or components shall not be used.
- ✓ In case of overheating, smoke, or abnormal smell, the power supply shall be immediately switched OFF.
- ✓ Report accidents as soon as possible to the department chair, department secretary, ECE technician, or the faculty in charge of the lab.

Rubrix for Mark Allocation

Criteria	Max Marks (100)	Excellent (Full Marks)	Good (75%)	Satisfactory (50%)	Needs Improvement (25%)
Preparation & Pre-lab Work	20	Demonstrates thorough understanding of the experiment objective, LPC2129 architecture, GPIO, LCD, interrupts, Timer0, Embedded C concepts and required components. Complete pre-lab preparation.	Demonstrates good understanding of the experiment with minor gaps in theory or procedure; pre-lab work mostly complete.	Demonstrates basic understanding of the experiment; pre-lab notes are incomplete.	Poor preparation; unable to explain the experiment objective or basic LPC2129 concepts; pre-lab work not prepared.
Algorithm & Flowchart / Circuit Diagram	15	Develops a clear and logically correct algorithm/flowchart and accurate LPC2129 circuit diagram. Correctly identifies ports, pins, LCD connections, LEDs, interrupt pins and timer-related connections as applicable.	Algorithm/flowchart and circuit diagram are mostly correct with minor errors in logic, connections or labeling.	Basic algorithm/flowchart or circuit diagram is provided but is incomplete or unclear.	Algorithm/flowchart or circuit diagram is missing or incorrect.
Program Code	10	Writes a correct, well-structured Embedded C program using appropriate LPC2129 registers, GPIO configuration, functions, delay routines, LCD routines, interrupt configuration and Timer0 programming as required.	Program is mostly correct with minor syntax or logical errors; code is understandable.	Program is partially correct and requires frequent guidance; limited understanding of the code.	Program is incorrect/incomplete; unable to explain the program logic.
Program Execution & Output	25	Successfully executes the program using LPC2129 and obtains the expected output, such as LED blinking, modular function operation, LCD message/scrolling/custom symbols, EINT0 interrupt response, Timer0 delays or four-road traffic-light sequence.	Program executes and expected output is obtained with minor corrections or assistance.	Partial output is obtained with frequent instructor assistance.	Program fails to execute or output is incorrect/missing.
Results & Interpretation	10	Records accurate results and provides clear technical interpretation of LED logic, LCD operation, interrupt response, Timer0 timing, delay generation or traffic-light sequence as applicable.	Results are correct with a brief and relevant explanation.	Results are recorded but interpretation is limited or partially correct.	Results are incorrect/missing and unable to explain the observed output.
Teamwork & Laboratory Etiquette	10	Works effectively with team members, handles LPC2129 kits, LEDs, LCDs and other laboratory equipment carefully, follows laboratory safety procedures and maintains discipline.	Works well with the team and follows laboratory procedures with minor issues.	Participates minimally and occasionally requires reminders regarding equipment handling or laboratory discipline.	Poor teamwork, careless handling of equipment or failure to follow laboratory safety and discipline.
Oral Viva / Understanding	10	Demonstrates excellent conceptual understanding of LPC2129, Embedded C, GPIO, registers, LCD interfacing, modular functions, EINT0, Timer0 and traffic-light control; answers confidently.	Answers most questions correctly and demonstrates good conceptual understanding.	Answers basic questions but lacks depth and clarity; requires prompting.	Unable to answer basic questions or explain the implemented experiment.

CONTENTS

Expt. No.	Date	Name of the Experiment	Page No.	Marks Awarded	Signature

Introduction to LPC2129 Microcontroller

The **LPC2129** is a single-chip **16/32-bit microcontroller** based on the **ARM7TDMI-S processor core**, designed for high-performance embedded applications. It integrates high-speed flash memory, SRAM, and a wide range of peripherals, making it suitable for real-time control and communication systems.

The device supports both **32-bit ARM instruction set** and **16-bit Thumb mode**, enabling efficient code execution and reduced program size. A built-in **memory accelerator module** and **128-bit wide memory interface** allow execution at high speed, supporting clock frequencies up to **60 MHz**.

Key Features

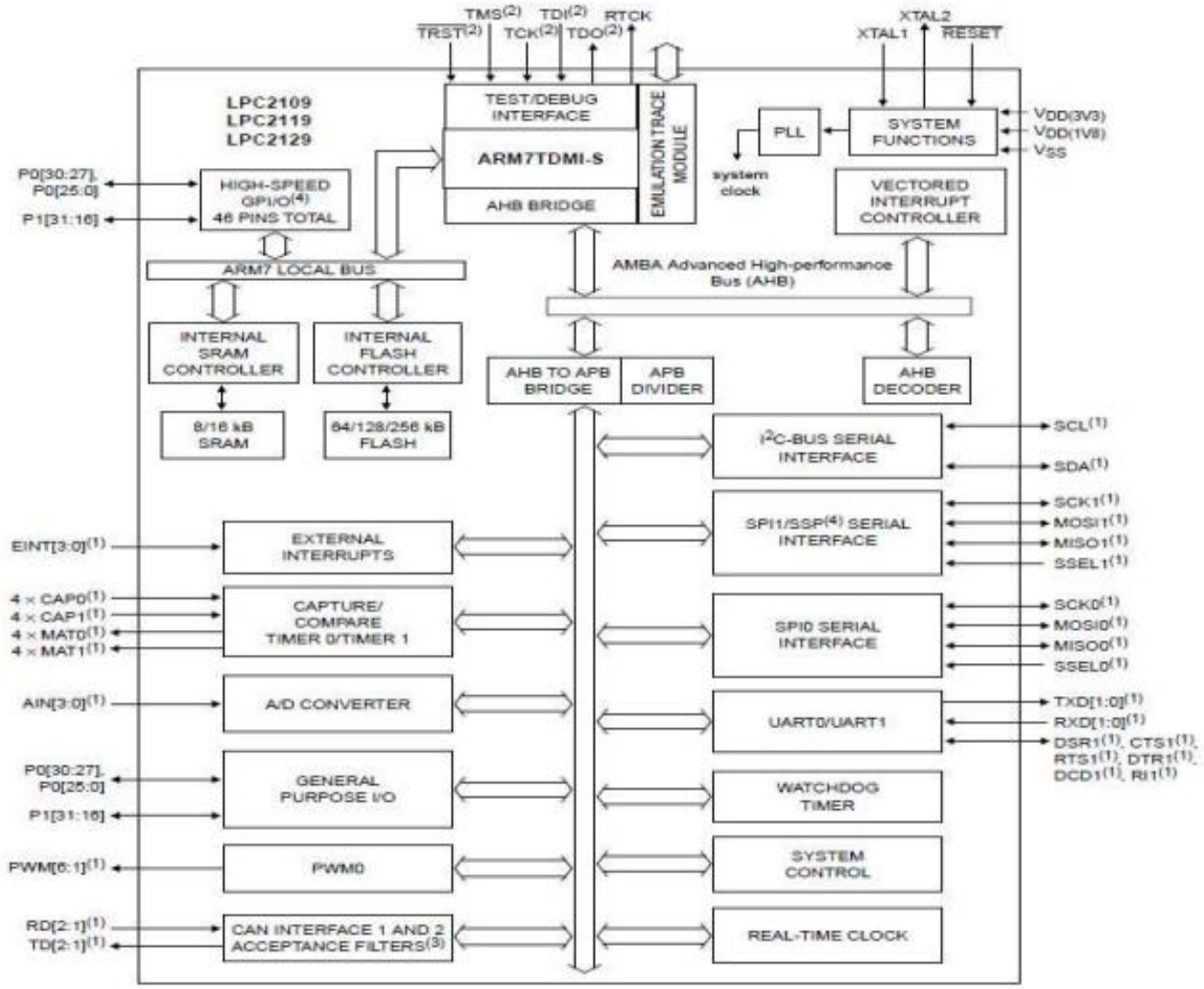
- **Processor Core**
  - ARM7TDMI-S core with real-time debugging and trace support
- **Memory**
  - 256 KB on-chip Flash memory
  - 16 KB SRAM
- **High-Speed Operation**
  - Maximum clock frequency up to 60 MHz using PLL
- **Analog Features**
  - 10-bit ADC with 4 channels
  - Fast conversion time (~2.44 µs)
- **Communication Interfaces**
  - Two UARTs (with auto-baud and fractional baud rate generator)
  - I2C interface (up to 400 kbit/s)
  - Two SPI interfaces / SSP
  - Two CAN interfaces with acceptance filters
- **Timers and PWM**
  - Two 32-bit timers with capture and compare features
  - PWM unit with 6 outputs
- **Interrupt Handling**
  - Vectored Interrupt Controller (VIC) with priority levels
- **GPIO**
  - Up to 46 fast GPIO pins (5V tolerant)
  - Supports external interrupts
- **Other Features**
  - Real-Time Clock (RTC)
  - Watchdog timer
  - On-chip oscillator (1 MHz to 30 MHz)
  - Power-saving modes: Idle and Power-down
  - Dual power supply (Core: 1.8V, I/O: 3.3V)

Architecture Overview

The LPC2129 follows a **von Neumann architecture** with a unified memory space for both data and instructions. It includes:

- CPU Core
- Memory Units (Flash & SRAM)
- Peripheral Modules
- System Control Block

Block Diagram



Block Diagram of LPC2129

The LPC2129 microcontroller consists of the following major functional blocks:

The LPC2129 block diagram includes:

- Central ARM7TDMI-S Core
- **Connected to:**
    - Flash (256 KB) and SRAM (16 KB)
    - AHB Bus → APB Bridge
- **Peripheral blocks:**
    - UART0, UART1
    - SPI / SSP
    - I2C Interface
    - CAN Controllers
    - ADC
    - PWM
    - Timers
    - RTC & Watchdog
- **GPIO Ports (P0, P1) at outer interface**
- **VIC and System Control at top level**

Applications of LPC2129

Due to its powerful features, LPC2129 is widely used in:

- Industrial automation systems
- Automotive control applications
- Medical equipment
- Communication gateways
- Data acquisition systems
- Embedded IoT applications

Advantages

- High processing speed with low power consumption
- Rich set of peripherals
- Reliable and stable for real-time applications
- Suitable for both beginner and advanced embedded development

Comparison: LPC2129 vs 8051 vs Arduino

Feature	LPC2129	8051	Arduino (ATmega328)
Architecture	32-bit ARM7	8-bit	8-bit
Speed	Up to 60 MHz	~12 MHz	16 MHz
Flash	256 KB	4 KB	32 KB
RAM	16 KB	128 B	2 KB
ADC	Yes (10-bit)	No	Yes
PWM	Yes	Limited	Yes
Applications	Advanced embedded	Basic control	Prototyping

Why LPC2129 in This Lab?

The LPC2129 microcontroller is used in this laboratory to:

- Understand ARM-based embedded system design
- Perform serial communication (UART, I2C, SPI)
- Interface sensors and external devices
- Develop real-time embedded applications using Embedded C

•

PROGRAM:

```
#include <LPC21xx.h>
void delay_ms(unsigned int ms)
{
   unsigned int i, j;
   for(i=0; i<ms; i++)
{
      for(j=0; j<12000; j++);
   }
}

int main()
{
   IODIR0 = 0xFF;
   while(1)
{
      IOCLR0 = 0xFF;
      delay_ms(1000);
      IOSET0 = 0xFF;
      delay_ms(1000);
   }
}
```

Expt. No: 1 Date:	I/O Programming using Microcontrollers

AIM:

To write and execute an Embedded C program using LPC2129 to blink 8 active Low LEDs connected to Port0 with a delay of 1 second.

HARDWARE & SOFTWARE REQUIRED:

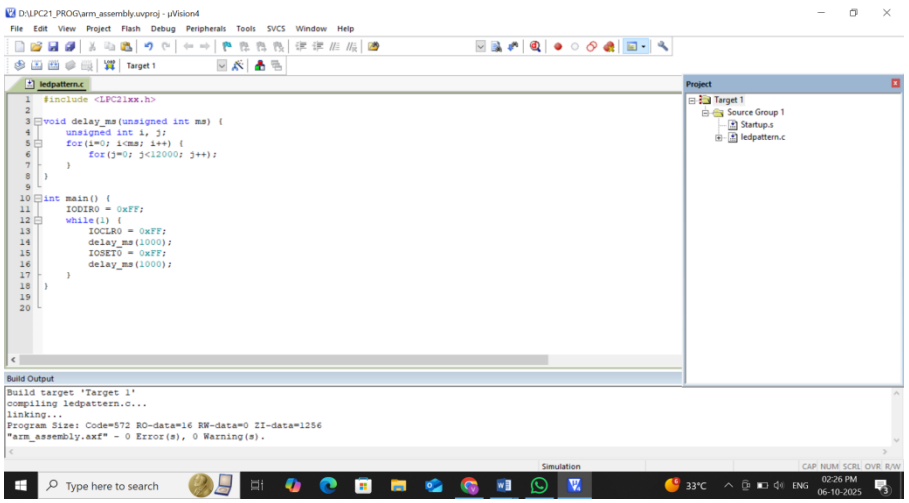
S.No.	Software Requirements	Quantity
1.	Keil µVision 4	1
2.	Proteus 8 Professional	1

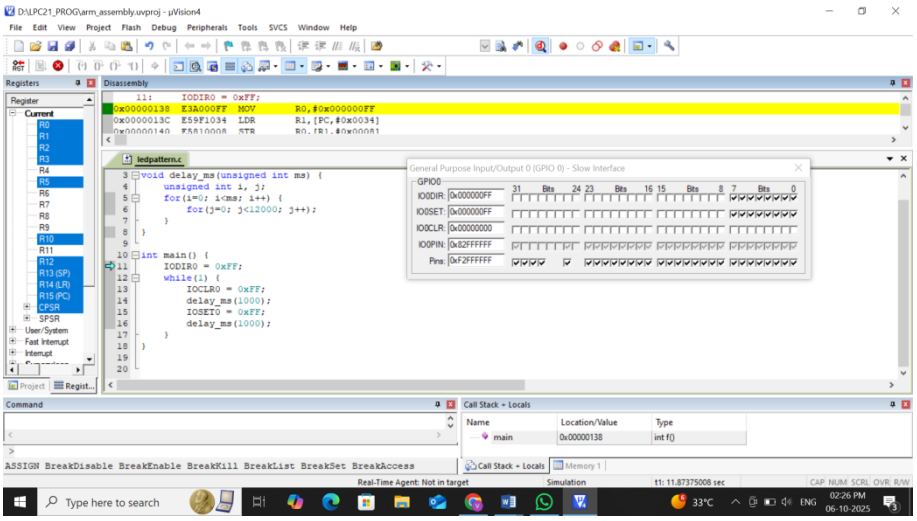
PROCEDURE:

1. Open **Keil µVision4** IDE.
2. Create a new project and select the **LPC2129** microcontroller.
3. Write the Embedded C program for LED blinking.
4. Compile the program and generate the .HEX file.
5. Open **Proteus Design Suite.**
6. Place the LPC2129 microcontroller and connect **8 LEDs (Active LOW)** to Port0 pins (P0.0 – P0.7).
7. Load the generated HEX file into the LPC2129 in Proteus.
8. Run the simulation and observe the LEDs blinking ON and OFF every 1 secon

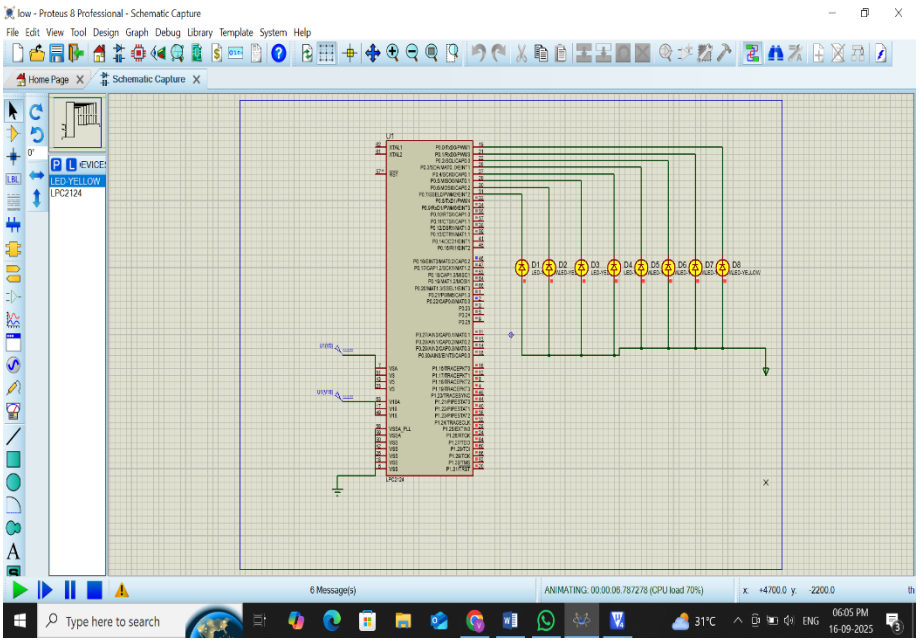
OUTPUT:

Keil Output:





Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM

```
#include <LPC21xx.h>
void delay_ms(unsigned int ms)
{
   unsigned int i, j;
   for(i=0; i<ms; i++)
{
      for(j=0; j<12000; j++);
   }
}

int main()
{
   unsigned int num;
   IODIR0 = 0xFF;
   while(1) {
      for(num=0; num<256; num++)
{
         IOSET0 = 0xFF;
         IOCLR0 = num;
         delay_ms(500);
      }
   }
}
```

Expt. No: 2 Date:	I/O Mapping and function based modular Programming

AIM:

To write and execute an Embedded C program using modular function in LPC2129 microcontrollers.

HARDWARE & SOFTWARE REQUIRED:

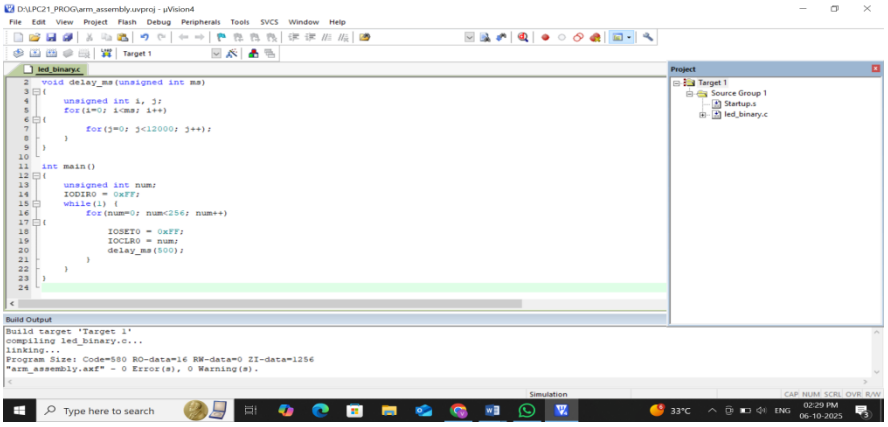
S.No.	Software Requirements	Quantity
1.	Keil µVision4	1
2.	Proteus 8 Professional	1

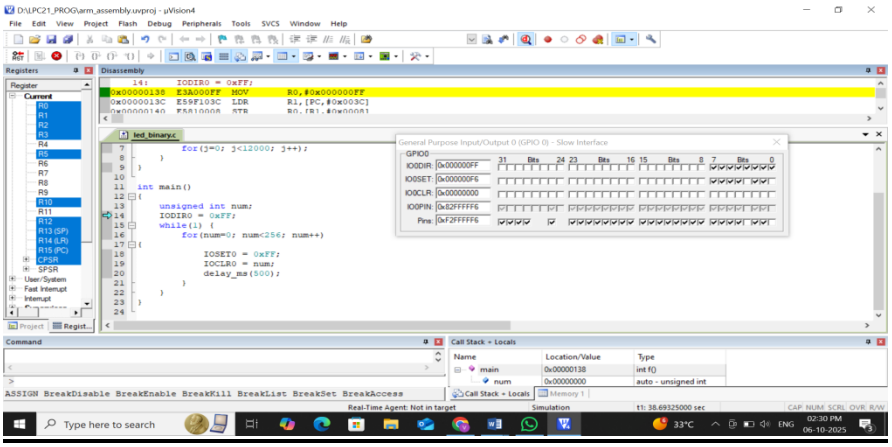
PROCEDURE:

1. Open Keil µVision4 IDE
2. Create a new project and select the LPC2129 microcontroller
3. Write the embedded c program to generate binary patterns from 0 to 255
4. Compile the program and generate the .HEX file
5. Open Proteus Design Suite
6. Place the LPC2129 microcontroller and connect 8 active low LEDs to Port0 pins (P0.0 – P0.7)
7. Load the HEX file into the LPC2129 in Proteus
8. Run the simulation and observe binary counting (0 to 255) on LEDs

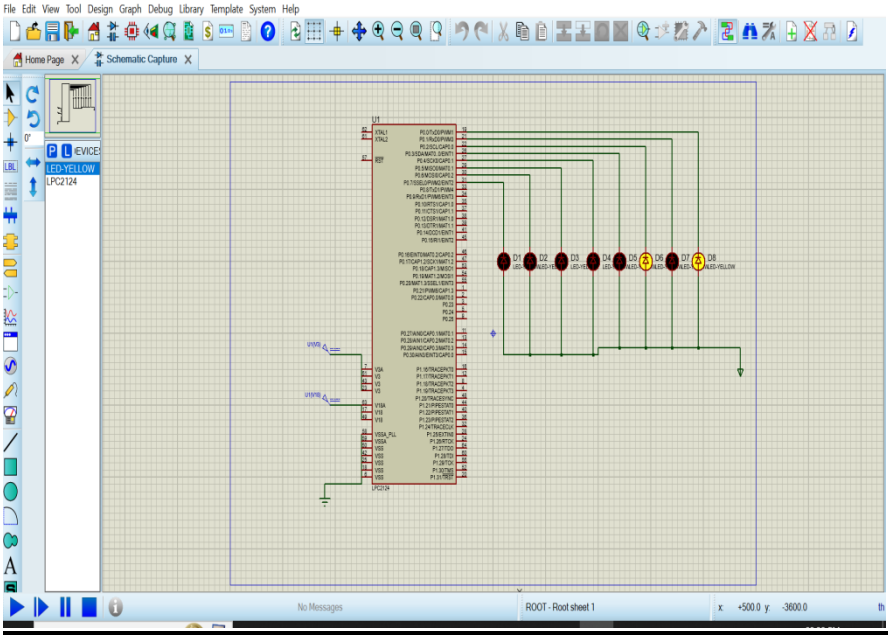
OUTPUT

Keil Output:





Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

Expt. No: 3 Date:	Programming to configuring GPIO

AIM:

To write and execute an Embedded C Program to display a messages on a 16×2 LCD using GPIO in LPC2129 microcontrollers.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Software Requirements	Quantity
1.	Keil µVision4	1
2.	Proteus 8 Professional	1

PROCEDURE:

1. Open **Keil µVision4 IDE**.
2. Create a new project and select the **LPC2129** microcontroller.
3. Write the Embedded C program to display messages on a **16×2 LCD** using GPIO.
4. Compile the program and generate the **HEX file**.
5. Open **Proteus Design Suite**.
6. Place the **LPC2129 and 16×2 LCD** and make the required GPIO connections.
7. Load the generated **.HEX file** into the LPC2129 in Proteus.
8. Run the simulation and observe the **messages displayed on the LCD**.

PROGRAM

```
#include <LPC21xx.h>

#define LCD_DATA 8
#define LCD_RS 16
#define LCD_RW 17
#define LCD_EN 18

void delay_us(unsigned int);
void delay_ms(unsigned int);

void WriteLCD(unsigned char);
void CmdLCD(unsigned char);
void InitLCD(void);
void CharLCD(unsigned char);
void StrLCD(char *);

int main()
{
   InitLCD();
   StrLCD("WELCOME TO KIOT");

   while(1);
}

void delay_us(unsigned int dlyUS)
{
   dlyUS *= 12;
   while(dlyUS--);
}

void delay_ms(unsigned int dlyMS)
{
   dlyMS *= 12000;
   while(dlyMS--);
}

void WriteLCD(unsigned char byte)
{
   IOCLR0 = 1 << LCD_RW;

   IOPIN0 = (IOPIN0 & ~(0xFF << LCD_DATA)) | (byte << LCD_DATA);

   IOSET0 = 1 << LCD_EN;
   delay_us(1);
   IOCLR0 = 1 << LCD_EN;

   delay_ms(2);
}
```

```
void CmdLCD(unsigned char cmd)
{
   IOCLR0 = 1 << LCD_RS;
   WriteLCD(cmd);
}

void InitLCD(void)
{
   IODIR0 |= (0xFF << LCD_DATA)
         | (1 << LCD_RS)
         | (1 << LCD_RW)
         | (1 << LCD_EN);

   delay_ms(15);

   CmdLCD(0x30);
   delay_ms(4);
   delay_us(100);

   CmdLCD(0x30);
   delay_us(100);

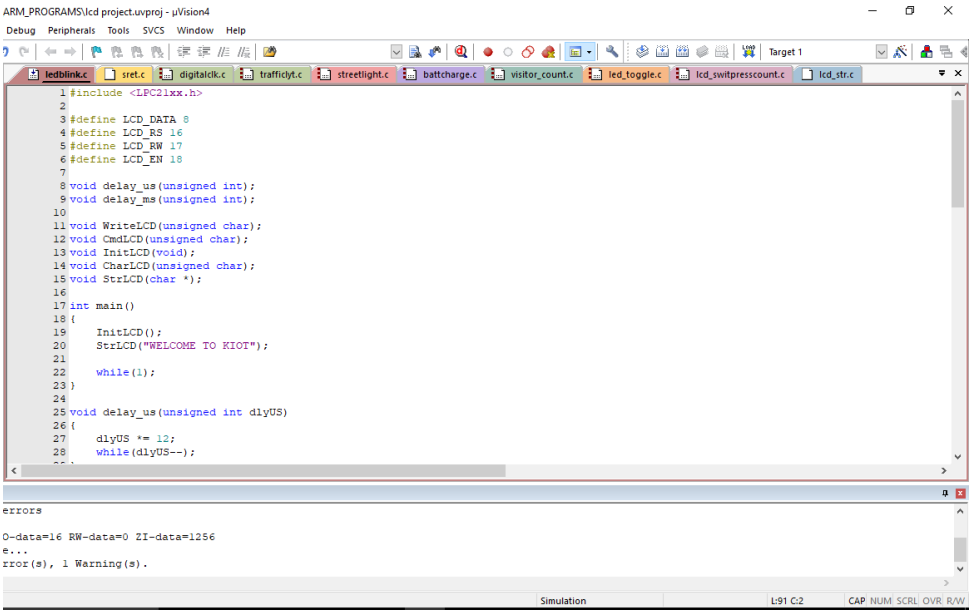
   CmdLCD(0x30);
   CmdLCD(0x38);
   CmdLCD(0x0C);
   CmdLCD(0x01);
   CmdLCD(0x06);
}

void CharLCD(unsigned char asciiVal)
{
   IOSET0 = 1 << LCD_RS;
   WriteLCD(asciiVal);
}

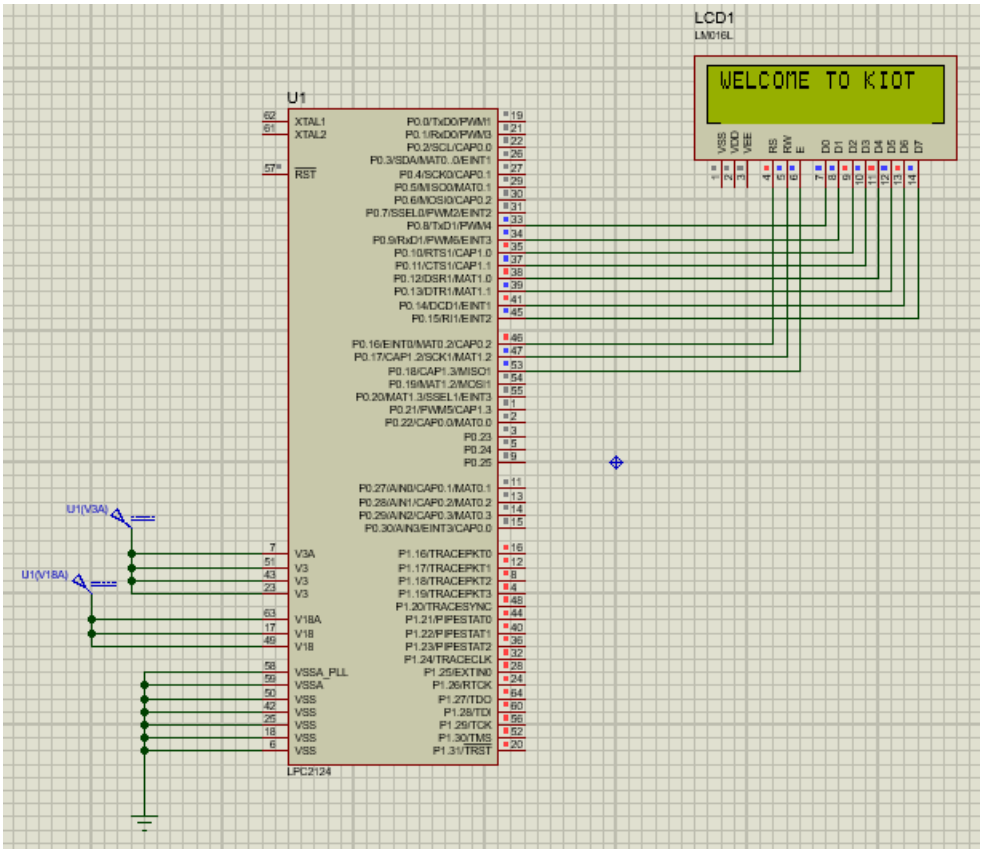
void StrLCD(char *s)
{
   while(*s)
   {
      CharLCD(*s++);
   }
}
```

OUTPUT

Keil Output:



Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM

```
#include<LPC21xx.h>
void ext_int0_isr(void)__irq
{
EXTINT = 0X01;
IOSET0 = 1<<0;
IOCLR0 = 1<<0;
VICVectAddr = 0;
}
int main()
{
unsigned int cnt = 0;
PINSEL1 = 0X00000001;
IODIR0 = 1<<0;
VICIntSelect = 0;
VICVectCntl0 =(0X20)|14;
VICVectAddr0 = ( unsigned long)ext_int0_isr;
EXTMODE = 0X01;
EXTPOLAR = 0X00;
VICIntEnable = 1<<14;
while(1)
{
cnt++;
}
}
```

Expt. No: 4 Date:	External Interrupt Programming

AIM:

To write an Embedded C Program to generate external interrupt 0 (EINT0) as a vectored IRQ using LPC2129.

HARDWARE & SOFTWARE REQUIRED:

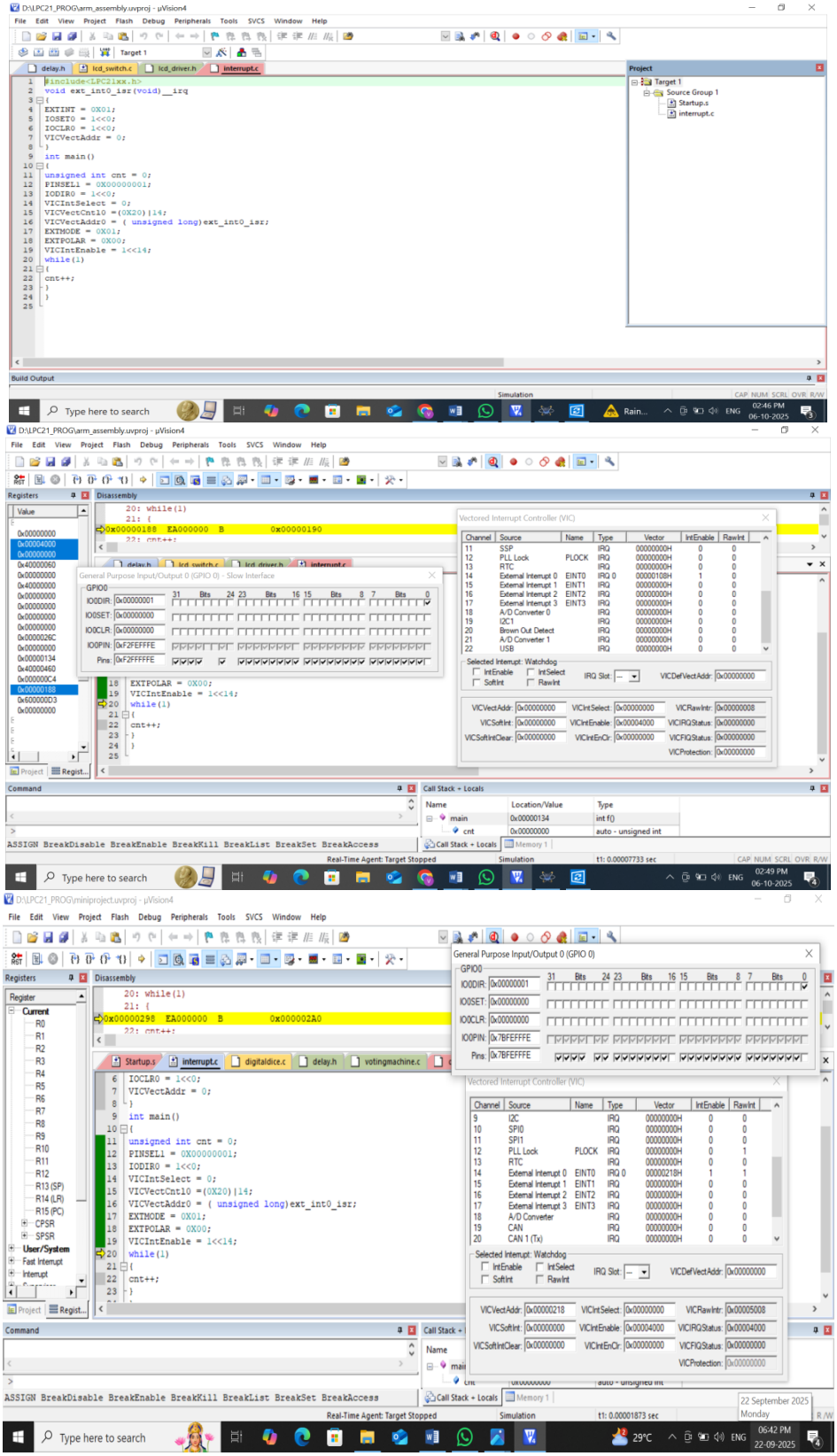
S.No.	Software Requirements	Quantity
1.	Keil µVision4	1

PROCEDURE:

1. Open Keil µVision4 software and create a new project.
2. Select LPC2129 microcontroller from the device database.
3. Write the C program to configure EINT0 as a Vectored IRQ.
4. Include necessary header files and initialize registers for interrupt.
5. Build and compile the program to generate the hex file.
6. Open the Debug session in Keil µVision4.
7. Use Logic Analyzer or Watch window to monitor the output on P0.0.
8. Simulate switch press by forcing EINT0 pin low and observe LED toggling in debug.

OUTPUT:

Keil Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

Expt. No:5 Date:	LCD Programming

AIM:

To write an Embedded C Program to scroll your name from left to right on a 16×2 alphanumeric LCD using LPC2129 microcontroller.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Software Requirements	Quantity
1.	Keil µVision4 IDE	1
2.	Proteus 8 Professional	1

PROCEDURE:

1. Connect the 16×2 LCD to LPC2129 (P0.0–P0.7 for data lines, P0.8 for RS, P0.9 for EN).
2. Include the header files lcd.h and delay.h in the program.
3. Initialize the LCD in 8-bit mode using LCD_INIT().
4. Store the message "Embedded" in a character array.
5. Use a loop to shift the cursor position from the first column to the 16th column minus message length.
6. At each step, clear the LCD, set the cursor using LCD_COMMAND, display the string using LCD_STR(), and insert delay using delay_milliseconds() for visible scrolling.
7. Repeat the loop continuously to achieve scrolling of the string across the LCD.

PROGRAM

```
#include <LPC21xx.h>

#define RS 16
#define EN 18

void delay_ms(unsigned int d)
{
   unsigned int i,j;
   for(i=0;i<d;i++)
      for(j=0;j<12000;j++);
}

void cmd(unsigned char c)
{
   IOCLR0 = 1<<RS;
   IOPIN0 = (IOPIN0 & 0xFFFF00FF) | (c<<8);
   IOSET0 = 1<<EN;
   delay_ms(2);
   IOCLR0 = 1<<EN;
}

void data(unsigned char c)
{
   IOSET0 = 1<<RS;
   IOPIN0 = (IOPIN0 & 0xFFFF00FF) | (c<<8);
   IOSET0 = 1<<EN;
   delay_ms(2);
   IOCLR0 = 1<<EN;
}

void display(char *s)
{
   while(*s)
      data(*s++);
}

int main()
{
   unsigned int i;

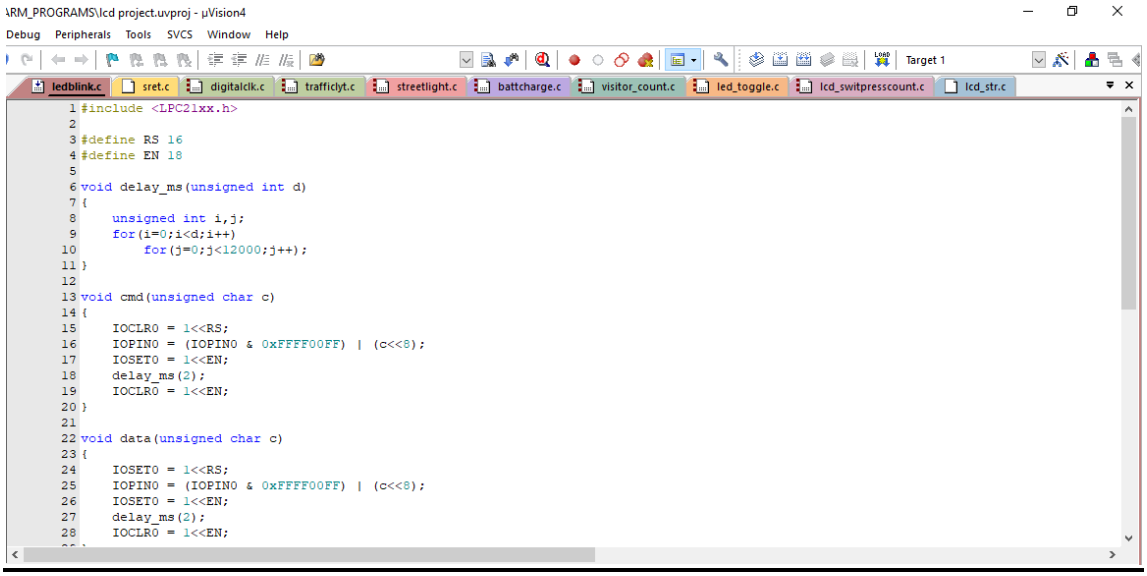
   IODIR0 = 0x0007FF00;

   cmd(0x38);
   cmd(0x0C);
   cmd(0x01);
   cmd(0x06);
```

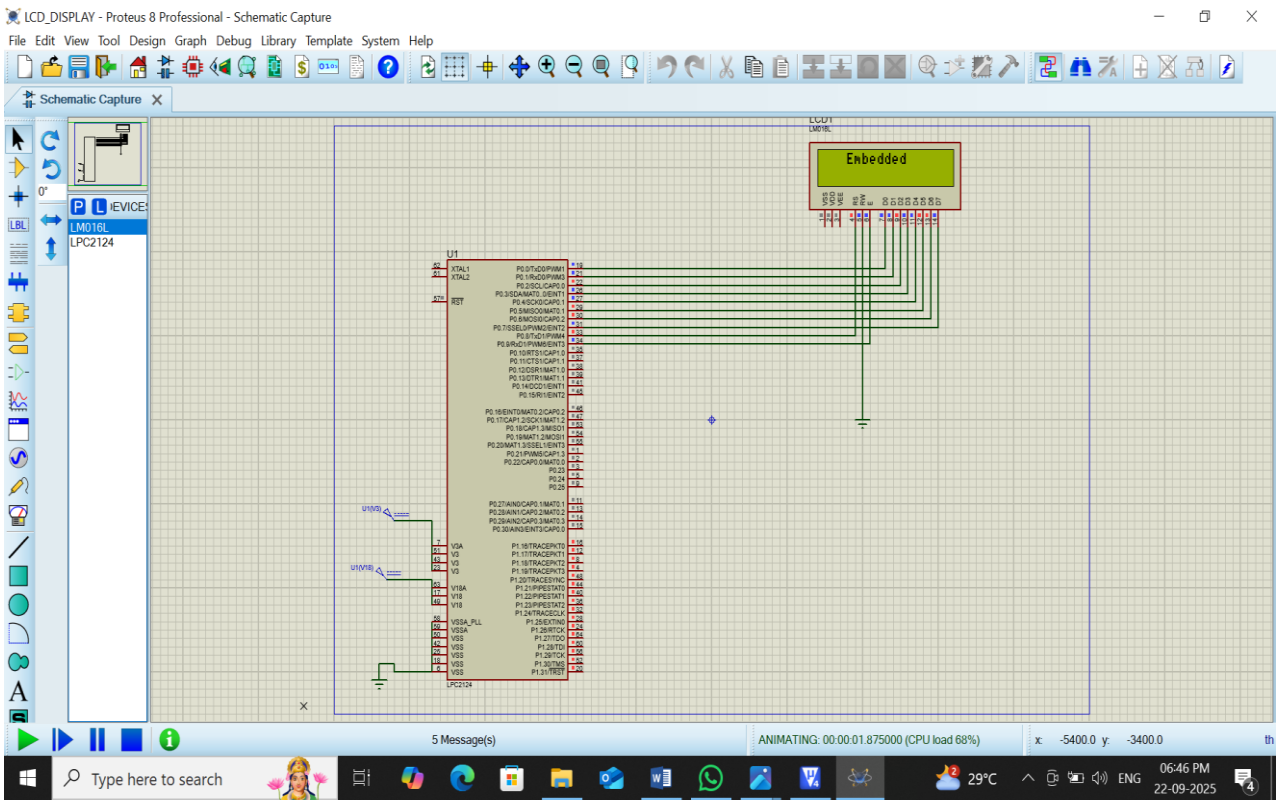
```
 while(1)
   {
      for(i=0;i<9;i++)
      {
         cmd(0x01);
         cmd(0x80+i);
         display("Embedded");
         delay_ms(150);
      }
   }
}
```

OUTPUT

Keil Output:



Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

Expt. No: 6 Date:	Serial Programming

AIM:

To write an Embedded C Program to create and display three custom symbols on a 16×2 alphanumeric LCD using LPC2129 microcontroller.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Software Requirements	Quantity
1.	Keil µVision4	1
2.	Proteus 8 Professional	1

PROCEDURE:

1. Connect the **16×2 LCD** to LPC2129 in 8-bit mode.
2. Connect **D0–D7 to P0.0–P0.7**, **RS to P0.8**, and **EN to P0.9**.
3. Define the byte patterns of the **three custom symbols** in separate arrays.
4. Initialize the LCD using `LCD_INIT()`.
5. Write the three custom symbols into different **CGRAM locations**.
6. Set the LCD cursor to the required position.
7. Display the three custom symbols using `LCD_DATA()`.
8. Compile the program in **Keil µVision4** and generate the HEX file.
9. Load the HEX file into **LPC2129 in Proteus** and run the simulation.
10. Observe the three custom symbols on 16x2 LCD

PROGRAM

```
#include <LPC21xx.h>

#define RS 16
#define EN 18

void delay_ms(unsigned int d)
{
   unsigned int i,j;
   for(i=0;i<d;i++)
      for(j=0;j<12000;j++);
}

void cmd(unsigned char c)
{
   IOCLR0 = 1<<RS;
   IOPIN0 = (IOPIN0 & 0xFFFF00FF) | (c<<8);
   IOSET0 = 1<<EN;
   delay_ms(2);
   IOCLR0 = 1<<EN;
}

void data(unsigned char c)
{
   IOSET0 = 1<<RS;
   IOPIN0 = (IOPIN0 & 0xFFFF00FF) | (c<<8);
   IOSET0 = 1<<EN;
   delay_ms(2);
   IOCLR0 = 1<<EN;
}

void lcd_init()
{
   cmd(0x38);
   cmd(0x0C);
   cmd(0x01);
   cmd(0x06);
}

void create_symbol(unsigned char address, unsigned char *pattern)
{
   unsigned char i;

   cmd(0x40 + address);
```

```
  for(i=0;i<8;i++)
      data(pattern[i]);
}

int main()
{
   unsigned char symbol1[8] =
   {0x04,0x0E,0x0E,0x0E,0x1F,0x00,0x04,0x00};

   unsigned char symbol2[8] =
   {0x0A,0x0A,0x0E,0x0E,0x0E,0x1F,0x00,0x00};

   unsigned char symbol3[8] =
   {0x04,0x0E,0x0A,0x1F,0x04,0x04,0x00,0x00};

   IODIR0 = 0x0007FF00;

   lcd_init();

   create_symbol(0x00,symbol1);
   create_symbol(0x08,symbol2);
   create_symbol(0x10,symbol3);

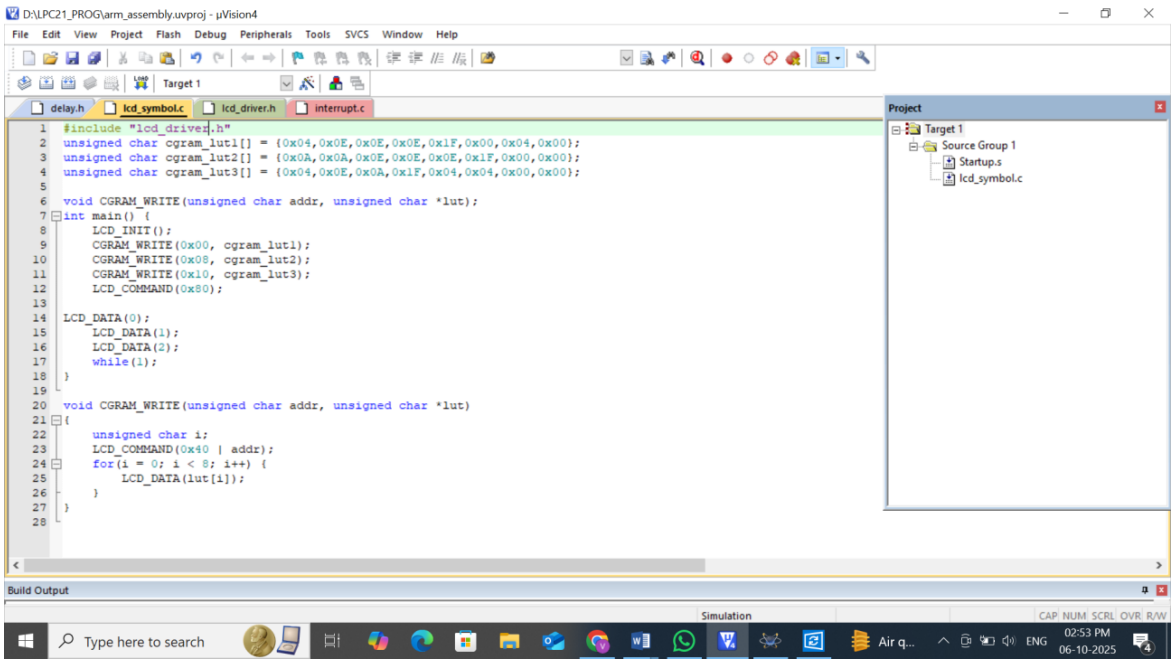
   cmd(0x80);

   data(0);
   data(1);
   data(2);

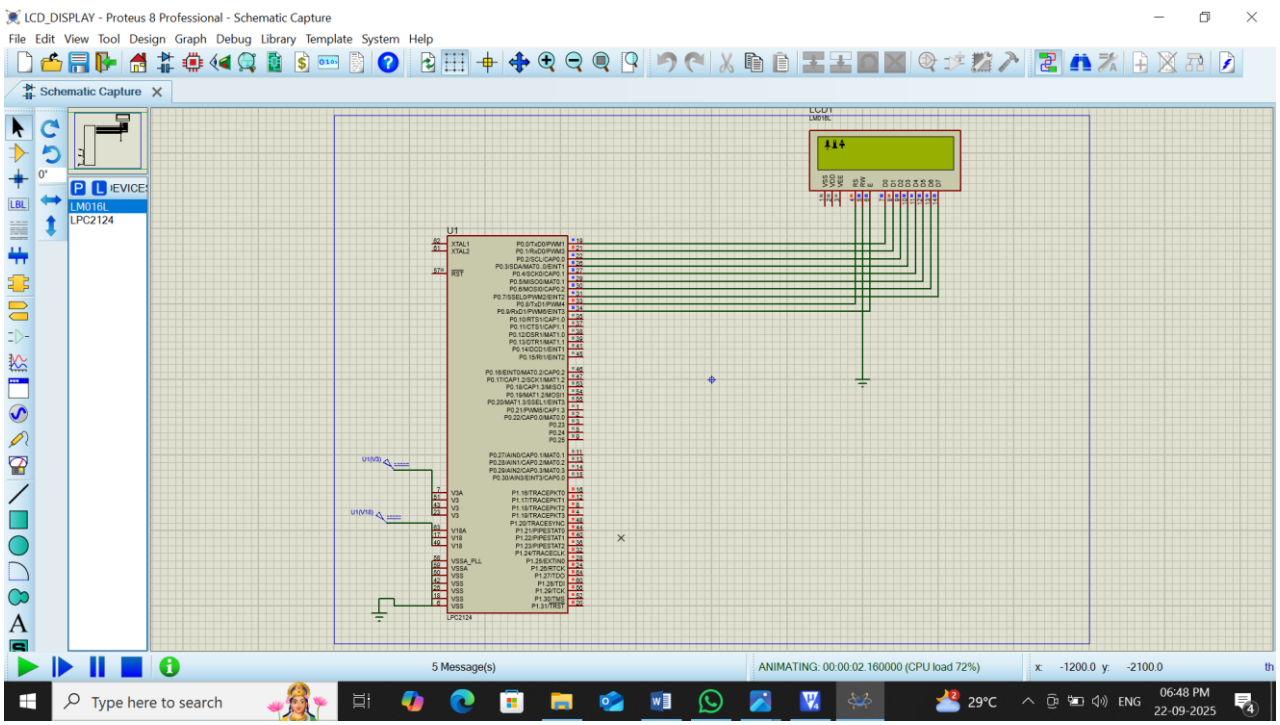
   while(1);
}
```

OUTPUT

Keil Output:



Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM

```
#include <LPC21xx.h>

void delay_sec(unsigned int sec)
{
   T0PR = 15000000 - 1;
   T0TCR = 0x01;

   while(T0TC < sec);

   T0TCR = 0x03;
   T0TCR = 0x00;
}

void delay_ms(unsigned int ms)
{
   T0PR = 15000 - 1;
   T0TCR = 0x01;

   while(T0TC < ms);

   T0TCR = 0x03;
   T0TCR = 0x00;
}

int main()
{
   IODIR0 = 0xFF;

   while(1)
   {
      IOSET0 = 0xFF;
      delay_sec(1);

      IOCLR0 = 0xFF;
      delay_ms(500);
   }
}
```

Expt. No: 7 Date:	Timer0-Based Delay Generation

AIM:

To demonstrate Timer0 based delay generation for microsecond, millisecond and second delays using LPC2129 Microcontroller.

HARDWARE & SOFTWARE REQUIRED:

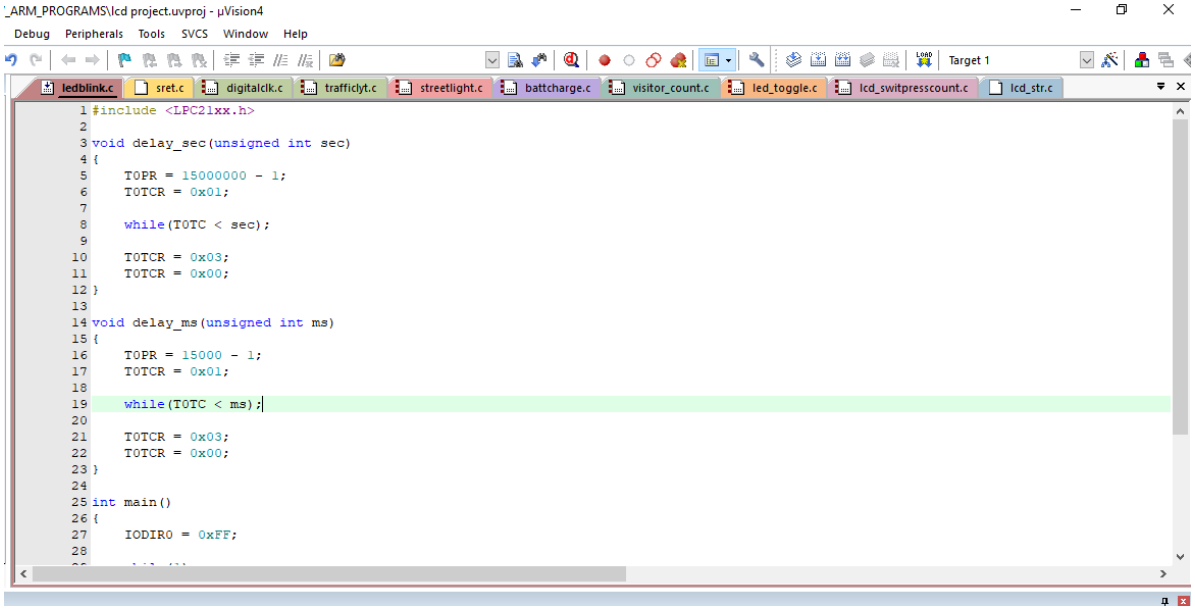
S.No.	Software Requirements	Quantity
1.	Keil µVision4	1
2.	Proteus 8 Professional	1

PROCEDURE:

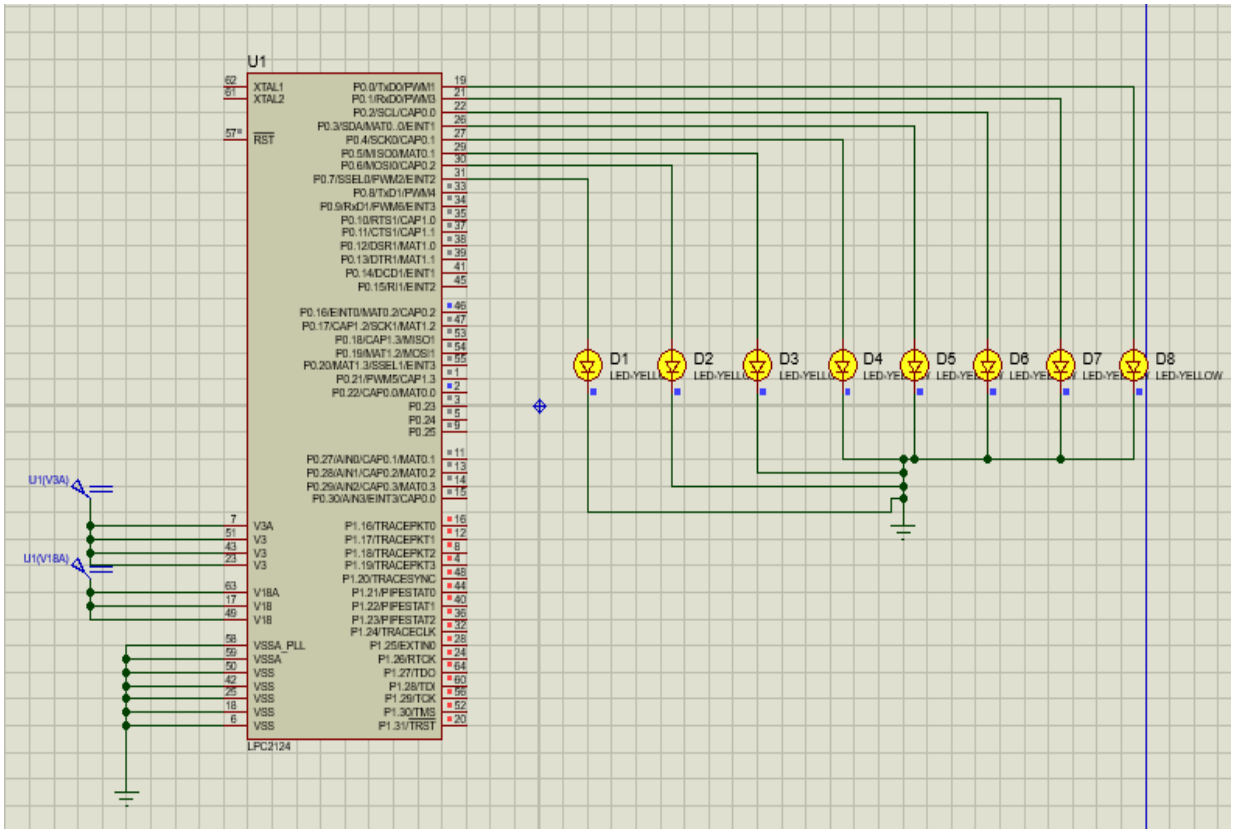
1. Open **Keil µVision4** and create a new project.
2. Select **LPC2129** microcontroller.
3. Write the Embedded C program using **Timer0** for delay generation.
4. Configure **Timer0** for millisecond and second delays.
5. Compile the program and generate the **HEX file**.
6. Open **Proteus 8 Professional** and place the **LPC2129 and 8 LEDs**.
7. Connect the LEDs to **P0.0–P0.7** through 330 Ω resistors.
8. Load the generated HEX file into LPC2129.
9. Run the simulation and observe the LEDs turning ON and OFF according to the Timer0 delay.

OUTPUT

Keil Output:



Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM

```
#include <LPC21xx.h>

#define S1R (1<<16)
#define S1Y (1<<17)
#define S1G (1<<18)

#define S2R (1<<19)
#define S2Y (1<<20)
#define S2G (1<<21)

#define S3R (1<<22)
#define S3Y (1<<23)
#define S3G (1<<24)

#define S4R (1<<25)
#define S4Y (1<<26)
#define S4G (1<<27)

void delay_ms(unsigned int ms)
{
   unsigned int i,j;

   for(i=0;i<ms;i++)
      for(j=0;j<6000;j++);
}

void All_OFF(void)
{
   IO1CLR = 0x0FFF0000;
}

void Road1_Green(void)
{
   All_OFF();

   IO1SET = S1G;
   IO1SET = S2R | S3R | S4R;
}

void Road1_Yellow(void)
{
   All_OFF();

   IO1SET = S1Y;
   IO1SET = S2R | S3R | S4R;
}
```

Expt. No: 8 Date:	Four-Road Traffic Light Control

AIM:

To design and implementation of a Four-Road Junction Traffic Light Control System using LPC2129 Microcontroller.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Software Requirements	Quantity
1.	Keil µVision4	1
2.	Proteus 8 Professional	1

PROCEDURE:

1. Open **Keil µVision4** and create a new LPC2129 project.
2. Write the Embedded C program to control the traffic lights for four roads.
3. Assign GPIO pins of LPC2129 for the **Red, Yellow and Green LEDs**.
4. Compile the program and generate the **HEX file**.
5. Open **Proteus 8 Professional** and place the LPC2129 and traffic-light LEDs.
6. Connect the LEDs to the assigned GPIO pins through resistors.
7. Load the generated HEX file into the LPC2129.
8. Run the simulation and observe the traffic lights changing in the required sequence for all four roads.

```
void Road2_Green(void)
{
   All_OFF();

   IO1SET = S2G;
   IO1SET = S1R | S3R | S4R;
}

void Road2_Yellow(void)
{
   All_OFF();

   IO1SET = S2Y;
   IO1SET = S1R | S3R | S4R;
}

void Road3_Green(void)
{
   All_OFF();

   IO1SET = S3G;
   IO1SET = S1R | S2R | S4R;
}

void Road3_Yellow(void)
{
   All_OFF();

   IO1SET = S3Y;
   IO1SET = S1R | S2R | S4R;
}

void Road4_Green(void)
{
   All_OFF();

   IO1SET = S4G;
   IO1SET = S1R | S2R | S3R;
}

void Road4_Yellow(void)
{
   All_OFF();

   IO1SET = S4Y;
   IO1SET = S1R | S2R | S3R;
}

int main(void)
{
   IO1DIR |= 0x0FFF0000;
```

```
    All_OFF();

    while(1)
    {
        Road1_Green();
        delay_ms(10000);

        Road1_Yellow();
        delay_ms(3000);

        Road2_Green();
        delay_ms(10000);

        Road2_Yellow();
        delay_ms(3000);

        Road3_Green();
        delay_ms(10000);

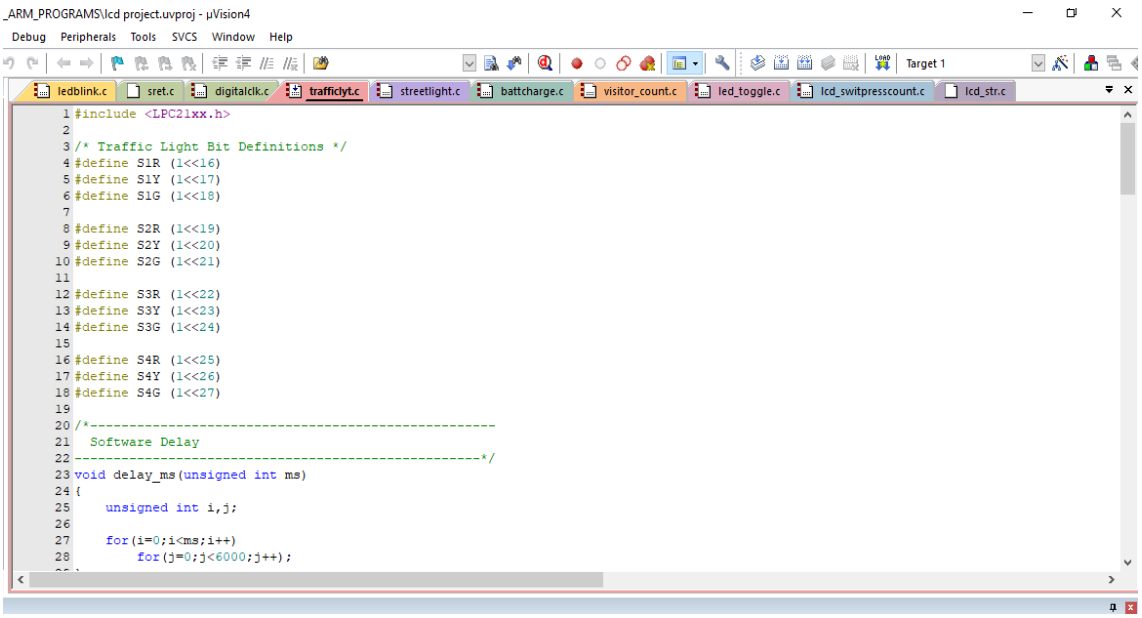
        Road3_Yellow();
        delay_ms(3000);

        Road4_Green();
        delay_ms(10000);

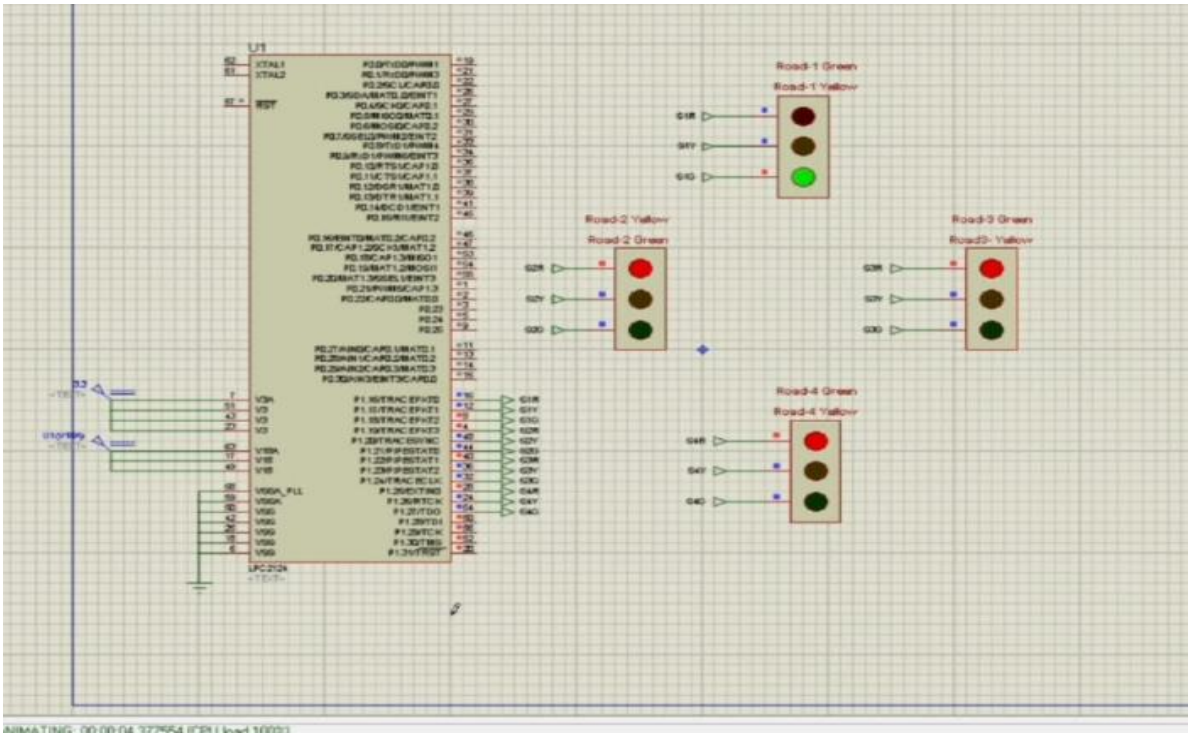
        Road4_Yellow();
        delay_ms(3000);
    }
}
```

OUTPUT:

Keil Output:



Proteus Simulation Output:



INFERENCE:

RESULT:

Assessment Parameters	**Max. Marks**	**Marks Obtained**
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

VIVA-VOCE Questions and Answer

A. LPC2129 and General

1. What is the LPC2129?

A single-chip 16/32-bit microcontroller from NXP based on the ARM7TDMI-S core. It has 256 KB flash and 16 KB SRAM, runs up to 60 MHz using the PLL, and has 2 UARTs, I2C, 2 SPI, 2 CAN, a 10-bit ADC, PWM and up to 46 GPIO pins.

2. What does ARM7TDMI-S stand for?

T = Thumb (16-bit instruction set), D = Debug, M = fast Multiplier, I = embedded ICE, S = Synthesizable.

3. Von Neumann vs Harvard architecture? Which does the LPC2129 use?

Harvard has separate memories and buses for code and data. Von Neumann uses one memory space for both. The LPC2129 uses von Neumann.

4. Difference between IODIR, IOSET, IOCLR and IOPIN?

- **IODIR:** sets pin direction (1 = output, 0 = input).
- **IOSET:** sets selected pins to 1.
- **IOCLR:** clears selected pins to 0.
- **IOPIN:** reads or writes the whole port value.

5. What is the VIC?

The Vectored Interrupt Controller. It manages interrupt priorities and gives the CPU the ISR address directly for vectored IRQs.

6. What is PINSEL?

The Pin Function Select register. It chooses whether a pin works as GPIO or as an alternate function (UART, I2C, EINT0 and so on).

B. GPIO and LED

7. What is an active-low LED? Which statement turns it ON?

The LED glows when the pin is 0. So IOCLR0 = 0xFF; turns the LEDs ON and IOSET0 = 0xFF; turns them OFF.

8. Why is while(1) used in embedded programs?

There is no OS to return to, so the main loop runs forever.

9. What is the drawback of a software delay loop?

It is inaccurate (depends on clock and compiler optimization) and it blocks the CPU. A hardware timer is better.

10. In the binary counter, why IOSET0 = 0xFF before IOCLR0 = num?

It first turns all LEDs OFF (active low). Then IOCLR0 = num turns ON only the LEDs whose bits are 1 in num, so the LEDs show the count 0 to 255.

11. What is modular programming?

Dividing a program into functions (such as delay_ms()), each doing one task. It makes code reusable and easier to debug.

C. LCD Programming

12. What are the LCD control pins?

- **RS:** register select (0 = command, 1 = data).
- **RW:** read/write (0 = write).
- **EN:** enable. A high-to-low pulse latches the data.

13. Why is 0x30 sent three times in LCD initialization?

It is the HD44780 reset sequence. It forces 8-bit mode reliably regardless of the power-on state.

14. What do the common LCD commands mean?

Command	Function
0x38	8-bit mode, 2 lines, 5x7 font
0x0C	Display ON, cursor OFF
0x01	Clear display
0x06	Entry mode, increment cursor
0x80	Cursor to line 1, position 0
0xC0	Cursor to line 2, position 0

15. How does cmd() differ from data()?

cmd() clears RS (command register). data() sets RS (data register). Both put the byte on the data pins and pulse EN.

16. Explain IOPIN0 = (IOPIN0 & 0xFFFF00FF) | (c<<8).

It clears only the 8 data bits (P0.8 to P0.15) and writes the new byte there, leaving all other pins unchanged.

17. How does the name scroll from left to right?

A loop runs i from 0 to 8. Each time it clears the LCD (0x01), sets the cursor to 0x80 + i, prints "Embedded" and waits 150 ms. The text shifts one position right each time. It stops at 8 because 16 columns minus 8 characters is 8.

18. What is CGRAM and how is a custom symbol created?

Character Generator RAM stores up to 8 user-defined 5x8 characters. Send command 0x40 + address, then 8 data bytes (one per row, lower 5 bits used). Each symbol takes 8 bytes, so addresses are 0x00, 0x08 and 0x10. Display with data(0), data(1), data(2).

19. Why send cmd(0x80) after writing CGRAM?

It returns the address pointer to DDRAM (the display area). Otherwise further data would be written into CGRAM again.

D. Interrupts

20. Interrupt vs polling?

Polling keeps checking a status and wastes CPU time. An interrupt notifies the CPU only when an event occurs, so response is faster and more efficient.

21. What does irq mean?

A Keil keyword that makes the compiler generate IRQ-mode entry and exit code

(register save and restore) for the ISR.

22. Explain VICVectCntl0 = 0x20 | 14 and VICIntEnable = 1<<14.

EINT0 is VIC channel 14. In VICVectCntl0, bit 5 (0x20) enables the slot and 14 selects the source. VICIntEnable = 1<<14 enables that interrupt.

23. Why is EXTINT = 0x01 written inside the ISR?

It clears the EINT0 interrupt flag. Otherwise the interrupt would trigger again and again.

24. Why is VICVectAddr = 0 written at the end of the ISR?

It tells the VIC the ISR is finished so it can update its priority hardware.

25. What do EXTMODE and EXTPOLAR do?

EXTMODE = 0x01 selects edge-sensitive mode. EXTPOLAR = 0x00 selects falling edge.

26. Compare IRQ vs FIQ.

FIQ has higher priority and its own banked registers, so it is faster. IRQ is the normal interrupt.

27. What are the rules for writing a good ISR?

Keep it short, avoid delays and printf, clear the interrupt flag, use volatile for shared variables, and set a flag so the main loop does the heavy work.

E. Timers, PWM and Watchdog (Expt 7)

28. Explain T0PR = 15000 - 1 and T0PR = 15000000 - 1.

With PCLK = 15 MHz, a prescale of 15000 gives 1 ms per count, so T0TC increments every millisecond. A prescale of 15,000,000 gives 1 second per count.

29. What do T0TCR = 0x01, 0x03 and 0x00 do?

0x01 enables the timer, 0x03 enables and resets it, and 0x00 stops it. The delay function resets the timer after each use.

30. Why use Timer0 instead of a software delay loop?

It is accurate, independent of compiler optimization, and the delay is set by hardware.

31. What is PWM and where is it used?

Pulse Width Modulation keeps the frequency fixed and varies the duty cycle. Used for motor speed, LED brightness and servo control. The LPC2129 has 6 PWM outputs.

32. What is a watchdog timer?

A timer that resets the system if software fails to refresh it in time. It recovers the system from hangs.

F. Traffic Light Controller

33. Which pins are used and why IO1DIR |= 0x0FFF0000?

P1.16 to P1.27 drive 12 LEDs (Red, Yellow, Green for 4 roads). IO1DIR |= 0x0FFF0000 makes only those pins outputs without changing the others.

34. Explain the sequence and why it is safe.

Each road gets green for 10 s then yellow for 3 s while the other three roads stay red, then the cycle moves Road 1, 2, 3, 4. Only one road is green or yellow at a time, so

there are no conflicts. All_OFF() clears all 12 LEDs before each new state.

35. How would you improve the traffic light program?

Use Timer0 for accurate delays, add an all-red clearance interval, add pedestrian signals, or use sensors for adaptive timing.

G. Embedded C Concepts

36. What is volatile and why is it important?

It tells the compiler a variable can change outside the program (hardware register or ISR flag), so every access must read memory. Without it the optimizer may cache the value and the loop may never exit.

```
volatile unsigned int flag = 0;
while(flag == 0);   // flag set in an ISR
```

37. How do you set, clear, toggle and test a bit?

```
reg |=  (1 << n);    // set
reg &= ~(1 << n);    // clear
reg ^=  (1 << n);    // toggle
if (reg & (1 << n))  // test
```

38. Why avoid printf and malloc in embedded code?

printf is large and slow. Dynamic memory can fragment and fail with no OS to recover, so static allocation is preferred.

39. How are hardware registers accessed in C?

Registers are memory-mapped, so they are accessed through fixed addresses with volatile pointers. This is how LPC21xx.h defines them.

```
#define IODIR0 (*(volatile unsigned long *)0xE0028008)
```

40. Embedded C vs standard C?

Same language, but Embedded C works directly with hardware registers, has tight memory and timing limits, usually has no OS or console, and uses extensions such as __irq.

H. Embedded Systems Fundamentals

41. Microprocessor vs microcontroller?

A microprocessor is only the CPU and needs external memory and peripherals. A microcontroller has CPU, memory, GPIO, timers and communication peripherals on one chip.

42. Hard vs soft real-time system

In a hard real-time system a missed deadline is a failure (airbag, ABS). In a soft real-time system it only lowers quality (video streaming).

43. RISC vs CISC

RISC uses simple fixed-length instructions, mostly single cycle (ARM). CISC uses complex variable-length instructions (x86, 8051).

44. Flash vs SRAM vs EEPROM

Flash is non-volatile and holds the program. SRAM is volatile and fast, holding variables and stack. EEPROM is non-volatile and byte-erasable, for small configuration data.

45. What are the stack and heap?

The stack holds local variables and return addresses. The heap is for dynamic allocation. Stack overflow is a common embedded bug.

46. What happens at reset on an ARM7?

The CPU jumps to the reset vector at address 0, runs the startup code that sets up stacks and clocks, and then calls main().

I. Communication Protocols (UART, I2C, SPI, CAN)

47. How does UART work? How is the baud divisor found?

The frame is a start bit (0), 8 data bits (LSB first), optional parity and a stop bit (1). It is asynchronous and uses TX, RX and GND. Divisor = PCLK / (16 x baud). For 15 MHz and 9600 baud this is about 97.

48. What is I2C?

A two-wire (SDA, SCL) synchronous multi-master, multi-slave bus with 7-bit addressing. It needs pull-up resistors and runs at 100 or 400 kbps. To read an EEPROM: START, address + W, memory address, repeated START, address + R, read data, NACK, STOP.

49. Compare UART, I2C and SPI.

Feature	UART	I2C	SPI
Wires	2 (TX, RX)	2 (SDA, SCL)	4 (MOSI, MISO, SCK, SS)
Clock	Async	Sync	Sync
Devices	Point to point	Many (addressed)	Many (chip select)
Speed	Low to medium	Medium	High

50. What is CAN and why is it used in vehicles?

Controller Area Network is a robust multi-master message-based bus using differential lines (CAN_H, CAN_L). Lower message ID wins arbitration, and it has CRC and automatic error handling. It suits vehicles for its noise immunity, error detection and low wiring. The LPC2129 has two CAN controllers.